Decision Support Databases 25-26

Notes

University of Pisa

M.Sc. in Data Science and Business Informatics

Contents

Chapter 1

Introduction

In modern society, organizations accumulate large amounts of data, stored and managed by computers via a DBMS (Database Management System). However, this data is often underutilized, and important decisions are still based on intuition rather than informed decisions derived from data analysis. Decision support information systems are specific types of information systems designed for summarizing large amounts of data in a form that is easy to interpret. Analysis performed by such systems goes beyond simple queries executed by traditional DBMSs on operational databases, which are optimized for on-line transaction processing. Organizations maintain a separate database, called data warehouse, specifically organized for decision support applications.

Data, Information, Knowledge

Data is a representation of facts, concepts, or instructions without context, which can be processed by computers.
Information is data (raw or in a condensed form) interpreted within a certain context.
Knowledge is information that expands the recipient's capability of understanding reality, allowing them to make predictions, decisions, and take actions.

Each organization has a set of specific goals to pursue, and to achieve these goals, it uses a **structure**, a set of **resources**, and a set of **processes** that transform resources into goods and services. Processes can be classified in three categories according to the **Anthony triangle**: **strategic activities** (for long-range planning), **tactical activities** (for short term planning and control), and **operational activities** (for day to day operations). For any process, information is a key resource; different organizations will have different information needs, depending on their structure, resources, and processes. Information systems are designed to satisfy these needs.

1.1 Information Systems

Information System

An **information system** is an organized collection of resources, people, and procedures finalized to collect, store, process, and communicate the information needed to support the on-going activities of an organization.

A computerized information system is a type of information system where information is stored and processed by a computer. From now on, we will simply refer to computerized information systems as information systems.
Information systems can be classified into three categories:

- **Operational**: data is organized in a database, which is managed by a traditional DBMS and interacted with using transactions. This type of system is used for day-to-day operations;
- **Decision Support**: data is organized in a dedicated data warehouse, managed by a specialized DBMS. This type of system is used for Business Intelligence applications, i.e., to search for useful insight to support decision making. DSS have been developed in the late 70's to help managers in making decisions of three types:
  - **Structured**: for which a well-defined decision-making procedure exists;
  - **Unstructured**: for which a well-defined decision-making procedure does not exists, so that the decision will only depend on manager experience;
  - **Semi-structured**: when the decision-making procedure is partly defined, so it also requires manager input.
- **Web-based**: for E-commerce, to bring the business to the Web.

1.2 Data Warehouses

Data Warehouse

A **data warehouse** is a subject-oriented, integrated, nonvolatile, and time-varying collection of data in support of management's decisions.

A data warehouse is a specialized type of database that stores data by subject, not by applications (*subject-oriented*), includes data from multiple heterogeneous sources and merged in a coherent form (*integrated*), is not changed interactively via transactions, but is updated periodically to add or delete data (*nonvolatile*), and it contains historical data that describes the state of the enterprise at different points in time, unlike operational databases where data is kept up-to-date (*time-varying*). A special type of data warehouse is the **data mart**, which is focused on data regarding only one subject and is usually smaller in size.

Three types of decision support can be provided. **Reports**, which are simple presentations of data in a structured format, **multidimensional analysis**, which also allows users to interactively explore data from different perspectives, and **data mining/exploratory data analysis**, where algorithms are used to automatically discover interesting patterns or relationships in data.

The reasons for separating operational databases from data warehouses are mainly two: first, complex data analysis queries would degrade the performance of operational DBMSs, which are optimized for on-line transaction processing; second, the two systems have different structures, contents, and uses, since operational databases do not store any historical data and do not integrate different sources. Traditional applications that use databases are called **OLTP** (**On-Line Transaction Processing**), while decision support applications that use data warehouses are called **OLAP** (**On-Line Analytical Processing**).

1.2.1 Data Warehousing Architectures

The process of bringing data from operational database sources into a single data warehouse is called **data warehousing**. In practice, three types of solutions are adopted.

Single-Layer Architecture There is only one layer of data managed by the operational system. The data warehouse is defined as a view on the operational database, but is not a physically separate component. This solution is very low-cost and easy to implement, and as such is often used by small organizations. Of course, it does not actually separate operational and analytical operations, so it is not suitable for large organizations.

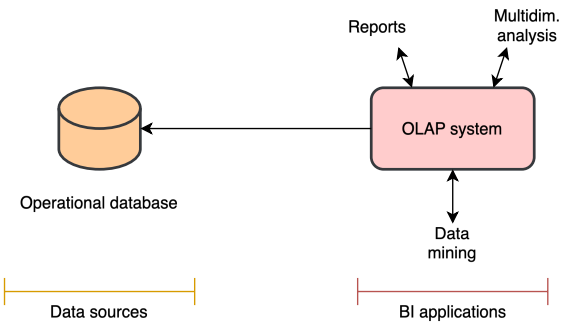

Figure 1.1: Single-Layer Architecture.

Two-Layer Architecture The data warehouse is a separate component from the operational database and is managed by its dedicated DBMS. The warehouse is loaded with data via **ETL** (**Extract, Transform, Load**) applications from the operational database and any other external source, which also brings this data in a consistent format and updates it periodically.

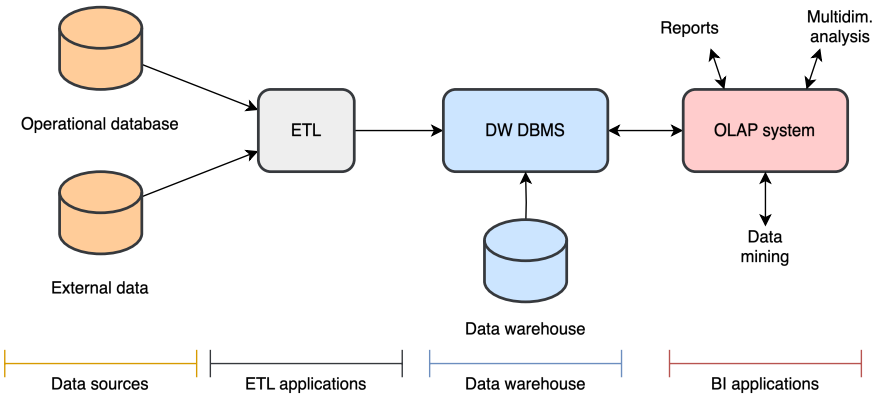

Figure 1.2: Two-Layer Architecture.

Three-Layer Architecture The third layer is called **data staging**. It contains data obtained via integration in the ETL process and prepares it to be loaded into the data warehouse, and may be implemented at different levels of complexity (it can be as simple as a set of files and as complex as a fully developed relational database). In this solution, extraction and integration is separated from reorganization and loading.

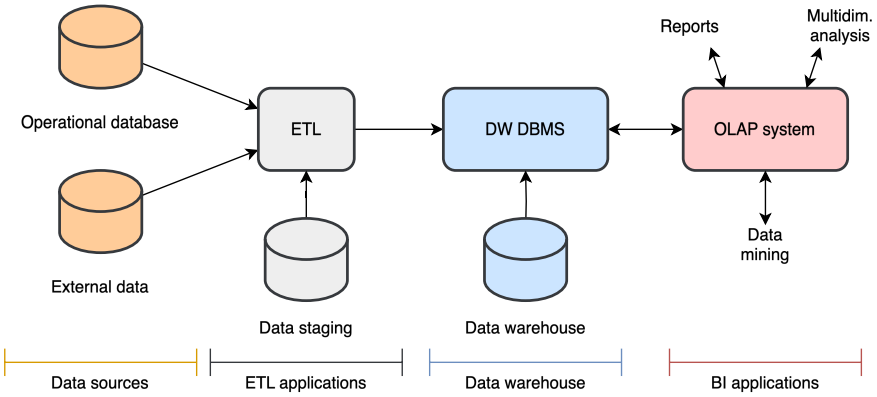

Figure 1.3: Three-Layer Architecture.

1.2.2 What to Model

Data must be organized taking into account how managers will use it to support their decisions about the performance of key business processes. More precisely, the relevant informa-

tion to model are:

- **Facts**: managers are interested in analyzing facts (i.e., observations) about the performance of a process;
- **Measures**: facts are accompanied by numerical attributes called measures (such as quantities, sizes, costs, revenues, etc.);
- **Dimensions**: managers think in terms of business dimensions, which give facts a context to make them meaningful. A dimension is often described by a set of attributes. If this is not true, it is called a *degenerate dimension*;
- **Metrics**: measures are analyzed via summaries called metrics, obtained by applying aggregation functions over facts by certain dimensions or combinations of dimensions. Common metrics are about economic performance, but when they relate to efficiency and quality they are called **Key Performance Indicators** (**KPI** for short).

  Aggregation functions are classified in three categories:

  - **Distributive**: *sum()*, *count()*, *max()*, *min()*.
  - **Algebraic**: *avg()*, *stddev()*, *var()*.
  - **Holistic**: *median()*, *mode()*, *rank()*.
- **Hierarchies**: managers are also interested in analyzing metrics at different levels of detail, by exploiting the fact that certain dimensions have a set of attributes that form a hierarchy (e.g., date can be broken down into year, semester, quarter, month, day).

A hierarchy always implies a functional dependency between the attributes involved.

Functional Dependency

Given a relation schema R and X, Y subsets of attributes of R, a functional dependency $X \to Y$ is a constraint that specifies that for every possible instance r of R and for any two tuples $t_1, t_2 \in r$, $t_1[X] = t_2[X]$ implies $t_1[Y] = t_2[Y]$.

In simpler terms, a functional dependency $X \to Y$ means that any time we know the value that a tuple takes in X we can uniquely determine the value it takes in Y. In the date example, there is the following functional dependency: day $\to$ month $\to$ quarter $\to$ semester $\to$ year. Note that in order for the hierarchy to hold, the day attribute must also include the specific month and year, the month attribute must include the year, and so on.

Chapter 2

Modeling for Operational Databases

This chapter discusses modeling methods and techniques for operational databases, introducing the main notation elements of the most common models.

2.1 Information Modeling

When a database system is created, it is based on some model that reproduces the essential characteristics of the real world information it is intended to represent. In general, models can be either *iconic*, if they retain physical characteristics of the entities they represent, or *symbolic*, if they use symbols to represent entities and their relationships. In the context of information systems, symbolic models are used.

Symbolic Model

A **symbolic model** is a subjective, formal representation of ideas and knowledge about some aspects of the real world, called domain of discourse, designed to serve a specific purpose.

A model simplifies reality by focusing on what is important for the task at hand, and uses a formal language to express this simplified representation.
Modeling in databases is broken down into three levels of abstraction:

- **Conceptual level**, in which the basic structure of the database is defined at an high-level, after analyzing the problem and considering any user requirement. Some examples of conceptual models are the **Entity-Relationship** (**ER**) model and the **Object Data Model** (**ODM**);
- **Logical level**, where the conceptual model is mapped to a more detailed description of the data structures, including all attributes and constraints. The model is still

independet from any specific DBMS. An example of logical model is the **Relational Data Model**;

- **Physical level**, where the physical data structures are defined for the elements in the logical model. The choices done at this level must take in consideration the features of the DBMS used to implement the database.

2.1.1 What to Model

The reality to be modeled is considered in terms of concrete knowledge, abstract knowledge, procedural knowledge, dynamics, and communications.

Concrete Knowledge Concrete knowledge refers to specific facts known of the reality to be represented. We can assume reality consists of *entities*, *properties*, and *relationships*.

Entity, Property, Relationship

An **entity** is anything for which certain facts should be recorded, independently of the existence of other entities.
A **property** is a fact about an entity which is not meaningful in itself, but only because it describes an entity of interest.
A **relationship** is a fact which correlates independent entities.

For example, we want to model the administration system of a library. Examples of entities can be a book, a bibliographic description, a registered user, or a loan record. Properties may be a user's name and address, or a book's title and author. Entities with the same properties are said to have the same **type**, and are classified in the same **collection**, or **entity set**. The books titled "1984", "Moby Dick", and "The Great Gatsby" are all part of the same entity set, and are all of type *book*. As for relationships, examples could be the relationship between a bibliographic description and one or more books (the same book may be owned multiple times by the library), or between a user and their loan records.

Abstract Knowledge Abstract knowledge concerns facts which impose certain restrictions on admissible values of concrete knowledge and on the way these values can evolve over time, or expresses rules on how to derive new information. These rules are called **integrity constraints**. In the library example, abstract knowledge could include rules about the data type of attributes, such that book properties *title* and *author* must be strings, while *length* must be a non-negative integer; another example may be the rule that a loan can only be borrowed for two weeks at a time, and must be associated with the *id* of a registered user; the age of a user can be automatically calculated by subtracting the *birthdate* from the current

date. Relationships can also have constraints that limit the possible correlated entities. The most important ones are **structural constraints**, which are:

- **Cardinality**, *one* or *many*, to specify how many entities of one collection can be associated with entities of another collection, e.g., a bibliographic description can be associated with *many* book copies, while a book copy is associated with only *one* bibliographic description, so the relationship is called *one-to-many*;
- **Participation**, *total* or *partial*, to specify whether an entity of one collection can have entities of another collection associated with it, e.g., a loan record must be associated with one registered user, so the participation from the loan record side is *total*; on the other hand, a registered user may have never loaned a book, so the participation from the registered user side is *partial*.

2.2 Data Models

To construct the conceptual model for an operational database, we define a **schema**, a collection of time-invariant definitions which model the structure of admissible data (including integrity constraints), and procedural knowledge (i.e., the elementary operations applied to concrete knowledge to cause changes). Different formalisms, each with a specific data model, can be used to define the schema.

Data Model

A **Data Model** is a set of abstraction mechanisms, with associated operators and implicit integrity constraints, used to define a database schema.

For operational databases, we will see the Object Data Model and the Relational Data Model, for the conceptual and logical levels respectively.

2.2.1 Object Data Model

The basic mechanisms of the ODM are:

- **Objects**: they are the equivalent of entities. An object is a software entity with an internal state represented by instance variables, and a behaviour described by a set of methods. Each object has its own identity that persists over time, even if its internal state change;
- **Object types**: just like an entity belongs to a type, an object belongs to an object type. An object type describes the state fields and the implementation of methods common to all objects of that type;

- **Classes**: a modifiable sequence of objects with the same type. They are the equivalent of entity sets. Each class introduces the definition of a type and a constructor for values of this type, as well as a name to denote the class itself;

- **Relationships**: classes have relationships with other classes, which are characterized by a cardinality and partiality. A relationship may also have attributes, or be recursive (i.e., a class has a relationship with itself);

- **Inheritance**: a mechanism that allows something to be defined by only describing how it differs from a previously defined one. It is distinct from subtyping, which is a relation between types such that when one type A is a subtype of another type B, then any operation applicable to type B can also applied to type A (although inheritance is one way to define subtypes). A type can be defined from a single supertype (*single inheritance*) or from several supertypes (*multiple inheritance*).

- **Class hierarchies**: similarly to types, classes can also be organized in hierarchies. If C_1 is a subset of C_2, then C_1 is a *subclass* of C_2, and this implies that:

  - The type of elements in C_1 is a subtype of the type of the elements in C_2;
  - The elements in C_1 are a subset of the elements in C_2, and C_1 inherits any relationship defined on C_2.

  When the same superclass is shared by two or more subclasses, two constraint can be defined: **overlap** and **covering**. A no overlap constraint specifies that two subclasses cannot share any element. In the absence of this constraint, the same object may belong to more that one subclass. A covering constraint specifies that the objects in the subclasses must collectively include all elements in the superclass. In the absence of this constraing, the superclass may have elements that do not belong to any subclass. When both constraints are present, we call the hierarchy a **generalization**.

The following figures illustrate a possible graphical notation used for ODM. There is no standard notation; the one used here is based on the notation used in UML.

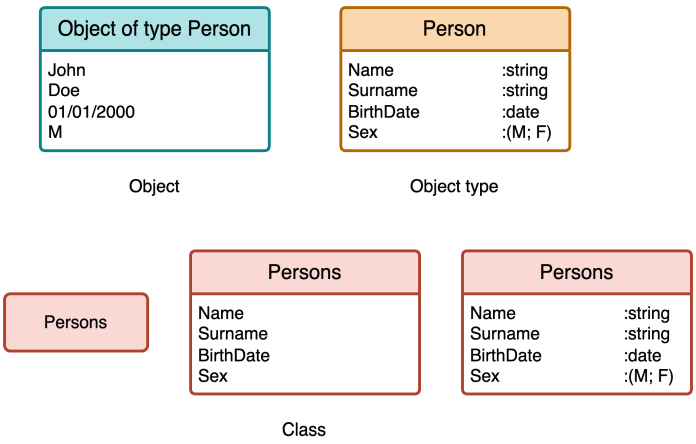

Figure 2.1: Graphical notation for objects, object types, and classes. Classes can be shown with different levels of detail.

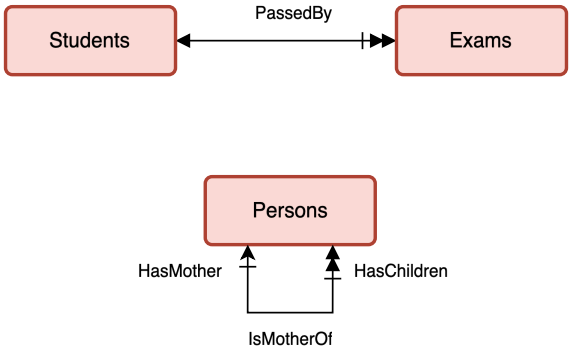

Figure 2.2: Simple relationships between classes. Each relationship is represented by an arc with a label. Multivalued relationships are represented with a double arrow. Optional relationships have a crossed line. Recursive relationships require labels on the ends of the arcs to clarify the meaning.

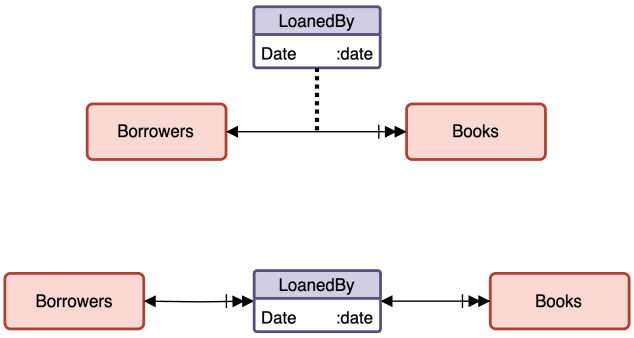

Figure 2.3: Relationships with attributes.

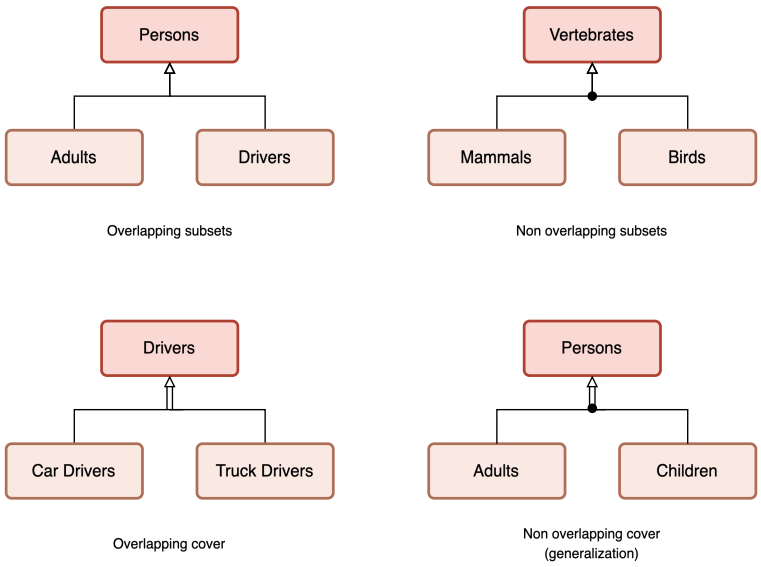

Figure 2.4: Class hierarchies.

2.2.2 Relational Data Model

The Relational Data Model describes databases in terms of sets of **tuples** (also called **records**), and associations among elements are expressed in terms of values of their attributes.

Relational Database

A **relational database** is described by a set of relation schemas $R : \{T\}$ defined as follows:

- **integers**, **floats**, **booleans**, and **strings** are primitive types.
- If $T_1, \ldots, T_n$ are primitive types, and $A_1, \ldots, A_n$ are distinct attribute names, then $T = (A_1 : T_1, \ldots, A_n : T_n)$ is a **tuple type** of degree n. The attribute order is unimportant.
- If T is a tuple type, then $\{T\}$ is a **relation type**.
- A **relation schema** $R : \{T\}$ is a variable R with a relation type.

For brevity, instead of writing $R : \{T\}$, we will use $R(T)$. Also, when the type of the attributes in a schema is unimportant, we will simply write it as $R(A_1, \ldots, A_n)$.

Tuple

A **tuple** $(A_1 := V_1, \ldots, A_n := V_n)$ of type $T = (A_1 : T_1, \ldots, A_n : T_n)$ is a set of pairs (A_i, V_i) with V_i of primitive type T_i.

Relation

A **relation** (or schema instance) is a finite set of tuples with the same type T.

A relation does not contain any duplicate tuples, since it is a set and not a multi-set. The order of relation elements and attributes is unimportant.
The following statements hold true:

- Two tuple types are equal if and only if they have the same degree, same attributes, and same type of the attributes with the same name.
- Two relation types are equal if and only if they have the same tuple types.
- Two tuples of the same type are equal if and only if they have the same set of attribute-value pairs.
- Two relations of the same type are equal if and only if they have equal tuples.

Key

A **key** for a relation is the minimal subset of attributes whose values identify a tuple. Out of all the possible keys, the database designer chooses a **primary key** (usually the shortest).

Relationships between relations are expressed using **foreign keys**, which take as values those of the primary key of another relation. For example, consider the two relations:

Students(*StudentCode, Name, City, BirthYear*)
Exams(*Subject, Candidate, Date, Grade*)

where attribute *StudentCode* is the primary key of *Students*, and the pair *Subject*, *Candidate* is the primary key of *Exams*. Since the attribute *Candidate* takes values that match those of *StudentCode*, it is a foreign key.

Relational Algebra Relational algebra is a set of operators on relations whose results are also relations. Expressions in relational algebra are called **queries**, and the standard language used by DBMSs to write queries is SQL (Structured Query Language): it is a declarative language, meaning that the user specifies what data to retrieve, and not how to retrieve it. There are six fundamental operations, plus additional operations that can be derived from them. These operations are:

- **Rename** $\rho_{A_1 \leftarrow B_1, \ldots, A_m \leftarrow B_m}(R)$: assigns a new name to attributes. $A_1, \ldots, A_m$ are attributes of relation R, and $B_1, \ldots, B_m$ are not. The result is a relation with type $\{(B_1 : T_1, \ldots, B_m : T_m)\}$, whose tuples are those of R with renamed attributes.

- **Project** $\pi_{A_1, \ldots, A_m}(R)$: selects a subset of attributes from the relation R, deleting any duplicate. The result is a relation with type $\{(A_1 : T_1, \ldots, A_m : T_m)\}$, whose tuples are those of R restricted to attributes $A_1, \ldots, A_m$.

- **Generalized project** $\pi_{exp_1 \text{ AS } B_1, \ldots, exp_n \text{ AS } B_n}(R)$: a combination of projection and renaming that also allows the output to contain the result of expressions. $exp_1, \ldots, exp_n$ are arithmetic expressions that involve constants and attributes of R, and $B_1, \ldots, B_n$ are attribute names. The result is a relation with type $\{(B_1 : T_{\text{exp}_1}, \ldots, B_n : T_{\text{exp}_n})\}$.

- **Select** $\sigma_{cond}(R)$: selects the tuples in R that satisfy the condition *cond*. This condition is a propositional logic expression $(\wedge, \vee, \neg)$ over comparison predicates $(\leq, <, \geq, >, =, \neq)$ and arithmetic expressions $(+, -, *, \div)$.

- **Grouping** ${}_{A_1, \ldots, A_n}\gamma_{f_1, \ldots, f_k}(R)$: returns a relation whose tuples are grouped by the values of the specified attributes, and applies the specified aggregate functions to the remaining attributes. $A_1, \ldots, A_n$ must be attributes of R, and $f_1, \ldots, f_k$ are aggregate functions (min, max, count, sum, average, etc.) calculated over one or more of the grouped attributes. The relation type of the result is $\{(A_1 : T_1, \ldots, A_n : T_n, f_1 : T_{f_1}, \ldots, f_k : T_{f_k})\}$.

- **Generalized grouping** ${}_{A_1, \ldots, A_n}\gamma_{f_1 \text{ AS } F_1, \ldots, f_k \text{ AS } F_k}(R)$: extends the grouping operation by allowing the renaming of attributes obtained via aggregation functions. The relation type of the result is $\{(A_1 : T_1, \ldots, A_n : T_n, F_1 : T_{f_1}, \ldots, F_k : T_{f_k})\}$.

- **Union** $R \cup S$: returns the union of the two sets of tuples in R and S, given that they have the same relation type.

- **Set difference** $R - S$: returns the set of tuples in R that are not also in S, given that they have the same relation type.

- **Product** $R \times S$: given T of type $\{(A_1 : T_1, \ldots, A_n : T_n)\}$ and S of type $\{(A_{n+1} : T_{n+1}, \ldots, A_{n+m} : T_{n+m})\}$ with disjoint set of attributes, the product operator returns a relation of type $\{A_1 : T_1, \ldots, A_n : T_n, A_{n+1} : T_{n+1}, \ldots A_{n+m} : T_{n+m}\}$, whose tuples are all possible cominations of tuples in R and S.

- **Join** $R \underset{A_i = A_j}{\bowtie} S$: combines tuples from R and S that have the same values for the attributes A_i of R and A_j of S. The two relations have type $\{(A_1 : T_1, \ldots, A_n : T_n)\}$ and $\{(A_{n+1} : T_{n+1}, \ldots, A_{n+m} : T_{n+m})\}$, and have a disjoint set of attributes. This operation is equivalent to calculating $\sigma_{A_i = A_j}(R \times S)$.

- **Intersection** $R \cap S$: returns the set of tuples that are present in both R and S, given that they have the same relation type.
- **Natural join** $R \bowtie S$: a special case of join that is only applicable when the two relations have attributes with the same name. It combines tuples in R and S that have the same values for all attributes with the same name in both.
- **Division** $R \div S$: let XY be the attributes of R, and Y the attributes of S. Then, the result of the division between R and S is a relation with attributes X such that

  $$W = \{w | \forall t \in S.(w \circ t \in R)\}$$

  where $\circ$ is the concatenation operator. In other words, it returns all the values of attributes X in R which appear in a tuple with all values of attributes Y in S.
  If $S \times W = R$, then $R \div S = W$.

There are also a series of **equivalence rules** to simplify expressions:

$$\pi_A(\pi_{A,B}(R)) \equiv \pi_A(R) \qquad \text{(Cascading of projections)}$$

$$\sigma_{cond_1}(\sigma_{cond_2}(R)) \equiv \sigma_{cond_1 \wedge cond_2}(R) \qquad \text{(Cascading of selections)}$$

$$\sigma_{cond_R \wedge cond_S}(R \bowtie S) \equiv \sigma_{cond_R}(R) \bowtie \sigma_{cond_S}(S) \qquad \text{(Commutativity of selection and join)}$$

$$R \bowtie (S \bowtie T) \equiv (R \bowtie S) \bowtie T \qquad \text{(Associativity of join)}$$

$$R \bowtie S \equiv S \bowtie R \qquad \text{(Commutativity of join)}$$

$$\sigma_{cond_X}({}_X\gamma_F(R)) \equiv_X \gamma_F(\sigma_{cond_X}(R)) \qquad \text{(Selection distributes over grouping)}$$

Chapter 3

Modeling for Data Warehouses

This chapter will illustrate the main design approaches and methodologies to create a data warehouse, and introduce the data models used to represent its structure.

3.1 Data Models

This section will introduce three models to define the structure of a data warehouse: for the conceptual level, the **Dimensional Fact Model** (**DFM**), while for the logical level, the **Multidimensional Relational Model** and the **Multidimensional Cube Model**.

3.1.1 Dimensional Fact Model

A data warehouse is based on a multidimensional model, i.e., a model that takes in consideration multiple business dimensions to allow the user to explore aggregated information across many possible dimensional combinations to get a full perspective on the data. The Dimensional Fact Model is a graphical conceptual model for data warehouses. The following are its basic elements.

Facts The most important abstraction mechanism of the conceptual model is the collection of facts, which are observations of the performance of a business model. Facts are modeled as rectangles divided in two parts: the top contains the fact name, while the bottom contains a set of **measures**. A measure is a numerical property of a fact that describes one of its quantitative aspects of interest for analysis. Facts that are not associated with any measure are called *factless fact*, although to keep terminology consistent, we will call them **measureless facts**. This type of fact is defined when there is only interest in counting its number of instances.
Facts can be of different nature:

- **Transaction facts** represent information on a specific event that occurred at a specific

point in time during the execution of a business process, e.g., a withdrawal on a bank account.

- **Periodic snapshot facts** represent the information on a series of events that have occurred over a period of time, e.g., the monthly summary of all transactions on a bank account.
- **Accumulating snapshot facts** represent the information on the lifetime of an evolving event that has a duration and a change over time, e.g., a mortgage application which is done in several steps (presenting the documents, undergoing bank review, approval/refusal, completion).

Also measures can be classified in differen types:

- **Additive/flow/rate measures** can be meaningfully aggregated with the function *sum* by any dimension. This is the most common type of measure. An example may be the number of products sold in a day.
- **Semi-additive/stock/level measures** can be meaningfully aggregated with the function *sum* by certain dimensions, but not all. These measures refer to a particular point in time and are used to record the state of the event: for example, the monthly inventory quantity of a certain product. We say that a measure is semi-additive with respect to a dimension D_1 if it cannot be aggregated with the function *sum* for groups of data with different values of D_1.
- **Non-additive/value-per-unit measures** cannot be aggregated with the function *sum* by any dimension. These measures are usually the result of ratios, so the only meaningful operation is counting the number of facts with a specific value. Other common non-additive measures are per-unit prices, measures of intensity (e.g., a temperature), or averages.
- **Calculated measures** are derivated from other measures. These are used to avoid having users perform these calculations, such as calculating revenue from product price minus discount.

Figure 3.1: Graphical representation of facts.

Dimensions Dimensions give facts a context. Graphically, they are represented by lines starting from the corresponding fact and ending with a circle. A dimension may also be described by a set of attributes, which are drawn as lines ending with a circle connected to the "main" circle of the dimension. Each attribute is also labeled with its unique name, and the same name should not be used for attributes of different dimensions.

In some cases, an interesting aspect to model is a hierarchical relationship between attributes. Using relational data model terminology, a hierarchy can be defined when there is a functional dependency between attributes: for example, if we define the dimension *Date*, with attributes *Day*, *Month*, *Year*, then we can define the hierarchy $Day \rightarrow Month \rightarrow Year$. The same attribute may even appear in multiple hierarchical structures for the same dimension: if we also add the attribute *Week*, then we can define $Week \rightarrow Year$ (this is because a week can overlap multiple months, so it does not belong to the previous dependency chain). A dimension without attributes is called **degenerate**.

Although not explicitly modeled at this level, the concept of changing over time is typically reported in the documentation, to be considered in the next phases. **Changing dimensions** are those dimensions whose attribute values may change over time. For example, a customer's address, phone number, or marital status; a student's enrollment status or major; a product's price or discount. There are four types of changing dimensions, the first three of which are **slowly changing dimensions**, and the last one being the **fast changing dimension**.

- **Type 1: overwriting the history**. When a dimensional attribute is updated, only the latest value is held in the data warehouse. This solution does not preserve the sequence of changes each attribute undergoes over time, potentially leading to a loss of context. For example, imagine a *Customer* dimension that contains various attributes, including the address. One customer changes residence from Rome to Milan, all sales that involved this customer will now consider the new Milan address, even if they occurred before the address change.

- **Type 2: preserving all history**. Both old and new values of the attribute are held, without changing the structure of dimensions. In practice, this means that each value change produces a new record in the data warehouse.

- **Type 3: preserving one or more versions of history**. If a dimensional attribute changes value, the structure of the dimension is extended with one or more additional attributes to keep track of different versions, as well as a timestamp that stores the date of the change. In the address change example, a new attribute *OldAddress* is created to keep the previous value with the Rome address, and the *address* attribute is updated with the Milan one. This solution is seldom used.

- **Type 4: fast changing**. The dimensional attribute changes very frequently (e.g., a status that is updated each few hours or minutes).

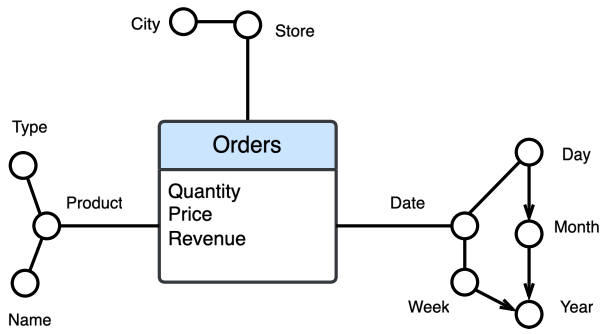

Figure 3.2: Graphical representation of dimensions.

The key steps to design a DFM schema are the following:

1. Gather requirements, identifying the key elements to insert in the model.
2. Identify the granularity of the fact, i.e., the level of detail at which the fact will be represented. Granularity determines which measures and dimensions will appear in the model. As a general rule, it is best to choose a fine grain, even if it increases the number of facts to be treated, since later on, aggregation functions can always reduce the level of detail.
3. Identify the fact measures. Not everything that is numeric is necessarily a measure.
4. Identify the dimensions and dimensional attributes. To discern dimensions in the original requirements analysis, it is useful to consider the classic 5W-1H questions.
5. If possible, identify any dimensional attribute hierarchy.

3.1.2 Multidimensional Relational Model

A conceptual model is transformed into a relational logical schema by applying a set of mapping rules. The main relational DBMS vendors provide OLAP servers that map operations on multidimensional data to standard relational operations on specialized relational DBMS to store and manage data warehouses. Such servers are called **ROLAP** (relational OLAP).

The basic elements of logical schemas are the **fact table** and the **dimension table**: a fact table stores all the measures of the corresponding fact and any necessary foreign keys to the dimension tables, while a dimension table stores all the attributes of the corresponding dimension, and eventually foreign keys to other dimension tables.

Schemas can follow three possible structures: **star schema**, **snowflake schema**, or **constellation schema**.

Star Schema A star schema consists of one fact table and a set of dimension tables, one per dimension in the conceptual model. Each dimension table has a single attribute as primary key, modeling a one-to-many relationship with the foreign key in the fact table. The primary key attribute is usually a sequentially increasing integer called **surrogate key**,

going from 1 to the total number of records in the table. Dates are also often assigned an integer surrogate key in the form *YYYYMMDD*, with the granularity of a day, while the remaining attributes simply specify the generic month/year/quarter/week/etc. the date corresponds to. Degenerate dimensions are either stored as attributes of the fact table, or in a **junk dimension** table, which lists all possible combinations of values of the degenerate dimensions. This structure is called "star" because the fact table is at the center and the dimension tables, which radiate out from it, look like the points of a star.

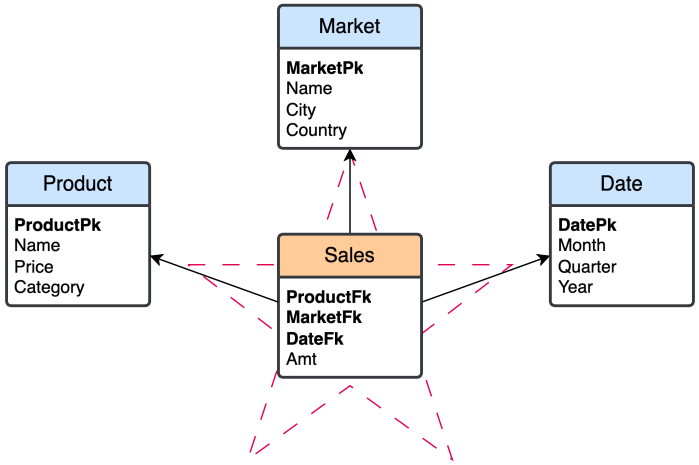

Figure 3.3: Example of a star schema.

Snowflake Schema In a snowflake schema, some dimension tables are normalized, splitting the data into additional tables. Its advantage is the reduction of redundancy in the dimension tables and therefore a reduction in storage space, however it is often a negligible improvement in comparison to the size of the fact table, and the increased complexity of certain queries that require hierarchies to be traversed. As a result, this schema is not as popular as the star schema.

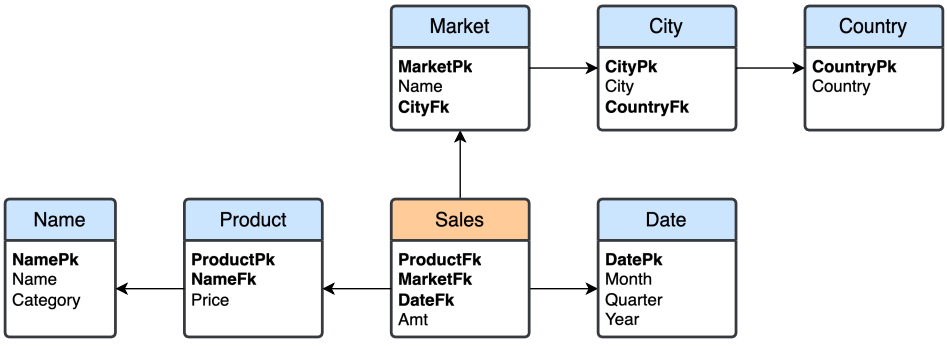

Figure 3.4: Example of a snowflake schema.

Constellation Schema A constellation schema is obtained by fusing multiple schemas with one or more dimensions in common. The result is a schema with several fact tables that share dimension tables.

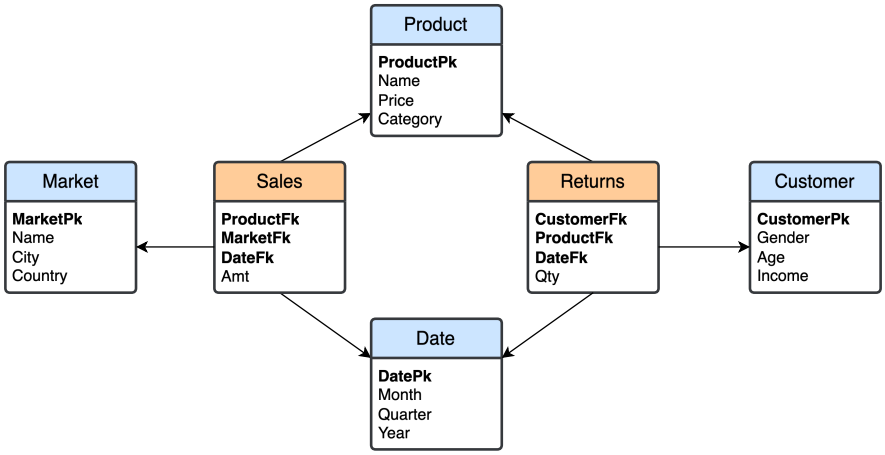

Figure 3.5: Example of a constellation schema.

3.2 Design Approaches

Data warehouse design can be approached from two perspectives: an analysis-driven one, or a data-driven one. In an **analysis-driven** (*bottom-up*, *metric pull*) approach, data marts are designed based on the data analysis the users want to perform, and then integrated in a single data warehouse. An advantage of this approach is that the focus it to provide what is really needed by the users, rather than what is available, and is usually cheap and fast to implement. A disadvantage is that the data necessary for analysis may not be available, and if a user is too tight on the requirements, the design may miss useful data that is available in the operational system. This approach was first proposed by R. Kimball.

In a **data-driven** (*top-down*, *data push*) approach, the data warehouse is designed around which data is currently available in the operational system, derivating the data marts afterwards. An initial design can help both users to discover new ways to use the data, and the designers to identify areas on which to focus more. A disadvantage is that it is a generally costly and time-consuming process, and without user involvement, the result may not be useful. This approach was first proposed by W. Inmon.

In practice, these two approaches are used together in different steps. The main phases of data mart design are requirements analysis, initial conceptual design, candidate conceptual design, final conceptual design, and logical design.

Requirements Analysis Requirements analysis consists in two sub-phases, requirements gathering and requirements specification. **Requirements gathering** analyzes the nature

and purpose of the business, identifying the important business processes and the business questions for which the data warehouse will be used to answer. This is done by directly interviewing the business users and data source system experts. **Requirements specification** uses the business questions found in the previous step to identify the facts, measures, metrics, and dimensions, as well as the granularity of data and dimensional attributes and hierarchies. The specific structure of the worksheet used to organize this information is the following:

- **Business process requirements**: each business question of each process is analyzed individually to find dimensions, measures, and metrics.

No.	Business questions	Dimensions	Measures	Metrics

- **Fact descriptions**: each identified fact is described in detail, including the preliminary dimensions and measures of the future data mart.

Grain, description, type	Preliminary dimensions	Preliminary measures

- **Dimensions**: the meaning of each dimension is described, together with its grain.

Name	Description	Granularity

- **Dimensional attributes**: each non-degenerate dimension is described by a number of attributes. It is good practice to use different names for attributes of different dimensions.

Attribute	Description

- **Dimensional hierarchies**: any hierarchy between attributes of the same dimension is described, indicating the direction and type (balanced, ragged, or recursive).

Dimension	Hierarchy description	Hierarchy type

- **Dimensional attribute changes**: if a dimension haschanging attributes, these are highlighted separately and annotated with the type (slowly changing of type 1, 2, 3, or fast changing/type 4).

Name	Changing attribute	Treatment of changes

- **Measures and metrics**: each fact measure has a description, an aggregability type (e.g., additive, semi-additive, etc.) and a flag that indicates if it is calculated from other measures.

Measure	Description	Aggregability	Calculated

- **Descriptive attributes of the facts**: contains each descriptive attribute of the identified facts.

Attribute	Description

- **Summary of dimensions and measures (optional)**: if the requirements concern multiple facts, a summary table is used to track which dimensions and measures are associated with which fact. The summary table will have one column per dimension, working as a binary flag to indicate belonging.

Dimension	Fact 1	...	Fact n

Measure	Fact 1	...	Fact n

Conceptual Design The initial, analysis-driven conceptual design focuses on the user requirements; the candidate, data-driven design is built by considering the data actually available in the operational database. A key sub-phase of this data-driven step is **entity classification**, in which the tables of the operational database are classified into three possible categories:

- **Transaction entities** are tables with records that represent events of interest that occur at a specific point of time and contain measurements or quantities that may be summarized.
- **Component entities** are tables related to a transaction entity via a *one-to-many* relationship. They define the details of each transaction event, so they are useful to answer the 5W-1H questions of a business event.
- **Classification entities** are tables related to a component entity via a (chain of) *one-to-many* relationship(s). They usually represent some hierarchy embedded in the data schema. Their more interesting attributes are collapsed into component entities to then define the dimensional attributes and hierarchies of the data mart.

The final design is obtained by comparing the previous two and finding a compromise between what is *useful* and what can be *delivered*.

Logical Design First, each final conceptual data mart design is translated into a relational schema, choosing the most appropriate structure (star or snowflake); then, if multiple data marts are designed, they are integrated in a single constellation schema, evaluating which facts and dimensions will be standardized and/or shared. Ideally, the translation must optimize query performance and maintain an intuitive user interface. Normalizing dimension tables, as mentioned in the previous sections, would typically interfere with both objectives.

Index